

Recursion Diagnostic Test

How to work out what a code fragment does: draw tables

The simplest and most reliable way to work out what a code fragment does (and to understand) is to draw a table. This is called tracing or dry running code, Let's work out what this fragment does when a function call `keepgoing(4)` is executed:

```
public static int keepgoing (int n)          // L1
{
    if (n<=1)                                // L2
        return 1;                            // L3
    else
    {
        int answer = 3 + keepgoing(n-2);      // L4
        System.out.println(answer);          // L5
        return answer;                        // L6
    }
}
```

Line L1 is executed as the function call storing the value 4 in parameter n. We then execute Line L2 which is a test `4<=1` is false so we jump to the else case.

keepgoing (4)						
LINE	n	n<=1 ?	answer	keepgoing(n-2) =?	return	printed
L1	4					
L2		4<=1? false				

We next, therefore, execute L4. It however calls a method so we pause and create a table for this new method call to work out what it does and get a returned result. However what it is calling is this method itself, so we pause this table and create a new table for the same method just with a different argument (a value stored in the n of this table of 2 - note that this is a different variable with its own storage space).

keepgoing (4)						
LINE	n	n<=1 ?	answer	keepgoing(n-2) =?	return	printed
L1	4					
L2		4<=1? false				
L4				keepgoing(2) =?		

keepgoing (2)						
LINE	n	n<=1 ?	answer	keepgoing(n-2) =?	return	printed

It has argument 2 so that is placed into n and the test done which again is false for this new value of n.

keepgoing (2)						
LINE	n	n<=1 ?	answer	keepgoing(n-2) =?	return	printed
L1	2					
L2		2<=1? false				

We again jump to line L4 and do another new call so start a third table.

keepgoing (2)						
LINE	n	n<=1 ?	answer	keepgoing(n-2) =?	return	printed
L1	2					
L2		2<=1? false				
L4				keepgoing(0) =?		

keepgoing (0)						
LINE	n	n<=1 ?	answer	keepgoing(n-2) =?	return	printed

Working on this new table, the value of n is 0 which means the test here is now true for this third variable called n. This time we do not do a recursive call, but just immediately return the answer of 1. That means this method call of keepgoing(0) is finished and we return the answer.

keepgoing (0)						
LINE	n	n<=1 ?	answer	keepgoing(n-2) =?	return	printed
L1	0					
L2		0<=1? true				
L3					1	

That means we can now return to the previous table for keepgoing(2) and fill in the answer to the call, and calculate the value stored in answer. Then we move to Line L5 for this call and print what is in answer (4). We then return the answer.

keepgoing (2)						
LINE	n	n<=1 ?	answer	keepgoing(n-2) =?	return	printed
L1	2					
L2		2<=1? false				
L4			4	keepgoing(0) =? 1		
L5						4
L6					4	

We can now go back to the original table and fill in the answer to its recursive call and continue with its execution

keepgoing (4)						
LINE	n	n<=1 ?	answer	keepgoing(n-2) =?	return	printed
L1	4					
L2		4<=1? false				
L4			7	keepgoing(2) =? 4		4
L5						7
L6					7	

We now have the value returned from the original recursive call.

What is printed

4
7

The final versions of the three tables generated are

keepgoing (4)						
LINE	n	n<=1 ?	answer	keepgoing(n-2) =?	return	printed
L1	4					
L2		4<=1? false				
L4			7	keepgoing(2) =? 4		4
L5						7
L6					7	

keepgoing (2)						
LINE	n	n<=1 ?	answer	keepgoing(n-2) =?	return	printed
L1	2					
L2		2<=1? false				
L4			4	keepgoing(0) =? 1		
L5						4
L6					4	

keepgoing (0)						
LINE	n	n<=1 ?	answer	keepgoing(n-2) =?	return	printed
L1	0					
L2		0<=1? true				
L3					1	

The results do not count for anything. This test is to allow the ECS401 team to diagnose if you need help to ensure they do not fall behind in learning to program. Understanding how for loops work is critical so if you have any problems we want to sort them out now. You must show all your working in the following. No correct table (exactly the same)... no marks.

1. What is printed after a method call `keepgoing2(5);`

```
public static int keepgoing2 (int n)          // L1
{
    if (n<=1)                                // L2
        return 3;                            // L3
    else
    {
        int answer = 5 + keepgoing2(n-2);    // L4
        System.out.println(answer);          // L5
        return answer;                       // L6
    }
}
```

Complete the dry run tables overleaf (you may not need all rows or all tables):

keepgoing2 (5)						
LINE	n	n<=1 ?	answer	keepgoing(n-2) =?	return	printed

keepgoing2 (3)						
LINE	n	n<=1 ?	answer	keepgoing(n-2) =?	return	printed

keepgoing2 (1)						
LINE	n	n<=1 ?	answer	keepgoing(n-2) =?	return	printed

Write exactly what is printed when the method is called in the box.

2. What is printed after a method call `keepgoing2(7);`

```
public static int keepgoing2 (int n)          // L1
{
    if (n<=1)                                // L2
        return 3;                            // L3
    else
    {
        int answer = 5 + keepgoing2(n-2);    // L4
        System.out.println(answer);         // L5
        return answer;                       // L6
    }
}
```

Complete the dry run tables overleaf (you may not need all rows or all tables):

keepgoing2 (7)						
LINE	n	n<=1 ?	answer	keepgoing(n-2) =?	return	printed

keepgoing2 (5)						
LINE	n	n<=1 ?	answer	keepgoing(n-2) =?	return	printed

keepgoing2 (3)						
LINE	n	n<=1 ?	answer	keepgoing(n-2) =?	return	printed

keepgoing2 (1)						
LINE	n	n<=1 ?	answer	keepgoing(n-2) =?	return	printed

Write exactly what is printed when the method is called in the box.

3. What is printed after a method call `keepgoing2(0);`

```
public static int keepgoing2 (int n)          // L1
{
    if (n<=1)                                // L2
        return 3;                            // L3
    else
    {
        int answer = 5 + keepgoing2(n-2);    // L4
        System.out.println(answer);         // L5
        return answer;                       // L6
    }
}
```

Complete the dry run tables (you may not need all rows or all tables):

LINE	n	n<=1 ?	answer	keepgoing(n-2) =?	return	printed

LINE	n	n<=1 ?	answer	keepgoing(n-2) =?	return	printed

Write exactly what is printed when the method is called in the box.